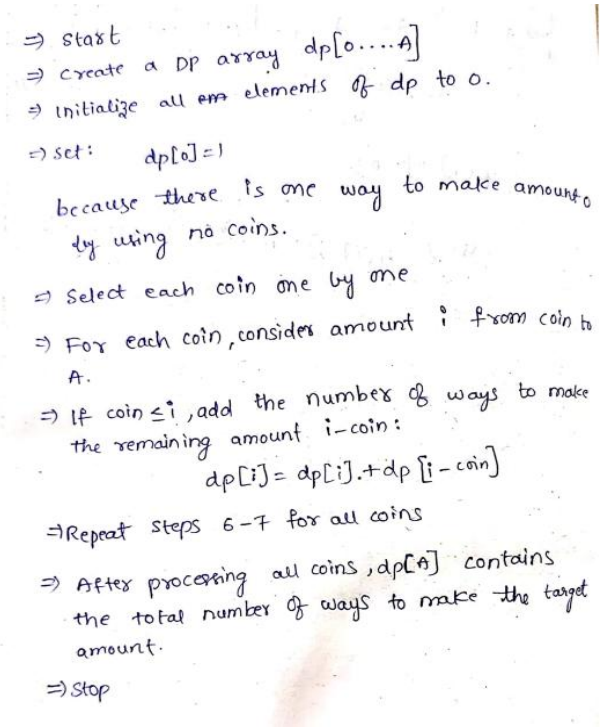
PRACTICAL-07

AIM: Implementation of making a change problem using dynamic programming.

COUNTING WAYS PROBLEM

ALGORITHM:



PROGRAM:

```
#include <iostream>
using namespace std;


int countWays(int coins[], int n, int amount) {
    int dp[amount + 1];
    for (int i = 0; i <= amount; i++)
        dp[i] = 0;


    dp[0] = 1;
```

```
for (int i = 0; i < n; i++) {
        for (int x = coins[i]; x <= amount; x++) {
            dp[x] += dp[x - coins[i]];
        }
    }
    return dp[amount];
}

int main() {
    int coins[] = {1, 2, 5};
    int n = sizeof(coins) / sizeof(coins[0]);
    int amount = 5;

    cout << "Number of ways to make " << amount << " = "
         << countWays(coins, n, amount) << endl;

    return 0;
}
```

OUTPUT:

```
Number of ways to make 5 = 4


...Program finished with exit code 0
Press ENTER to exit console.
```

TIME COMPLEXITY:

Total ways

```
for (int i=1 ; i<=n ; i++)          // 1 , n+1 , n
{
  for (int j=1 ; j<=w ; j++)        // n , n(w+1) -> nw
  {
    if (coins[i]>j)                 // -> nw
    {
      A[i][j] = A[i-1][j] ;         // -> nw
    }
    else
    {
      A[i][j] = A[i-1][j] + A[i][j-coins[i]];
    }
  }
}
```

n = no. of coins type, w = target amount

$T(n,w) = 1 + (n+1) + n + n + n(w+1) + nw + nw + nw$

$= 4nw + 4n + 2$

$= O(nw)$ → in worst case

→ Average case : The loop still runs for every coin and every amount

$T(n,w) = O(nw)$

→ Best case : The complete DP table is calculated the algorithm does not stop early

$T(n,w) = O(nw)$

MINIMUM WAYS PROBLEM

ALGORITHM:

⇒ Start
⇒ Create a DP array dp[0.....amount]
⇒ Set dp[0] = 0 because 0 coins are needed to make amount 0.
⇒ Initialize all other dp[i] values to ∞ (infinity)
⇒ For each amount i from 1 to amount:
  * Consider every available coin
  * If coin ≤ i calculate:
    dp[i-coin] + 1
  * Take the minimum value:
    dp[i] = min(dp[i], dp[i-coin]+1)
⇒ Repeat until the target amount is reached.
⇒ If dp[amount] is still ∞, the amount cannot be formed.
⇒ otherwise, dp[amount] is the minimum number of coins
⇒ Stop

PROGRAM:

```
#include <iostream>
#include <climits>
using namespace std;

int minCoins(int coins[], int n, int amount) {
    int dp[amount + 1];

    for (int i = 0; i <= amount; i++)
        dp[i] = INT_MAX;
    dp[0] = 0;
```

```
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1);
            }
        }
    }
    return (dp[amount] == INT_MAX) ? -1 : dp[amount];
}

int main() {
    int coins[] = {1, 5, 6, 9};
    int n = sizeof(coins) / sizeof(coins[0]);
    int amount = 10;
    int result = minCoins(coins, n, amount);
    if (result == -1)
        cout << "Change cannot be made for " << amount << endl;
    else
        cout << "Minimum coins needed for " << amount << " = " << result << endl;
    return 0;
}
```

OUTPUT:

```
Minimum coins needed for 10 = 2


...Program finished with exit code 0
Press ENTER to exit console.
```

TIME COMPLEXITY:

```
for (int i=1; i<=n ;i++)                          n+1
{  for (int j=1 ;j<=w;j++)                        n(w+1)  → nw
   { if (coins[i]>j)                              → nw
     { A[i][j] = A[i-1][j]                        → nw
     }
     else
     {
       A[i][j] = min(A[i-1][j], 1+ A[i][j-coin[i]])
     }
   }
}
```

$T(n,w) = 1+(n+1)+n+n+n(w+1)+nw+nw+nw$

$= 4nw + 4n + 2$

$= O(nw)$ → in worst case

⇒ Average case : The loops runs for every coin and every amount

$T(n,w) = O(nw)$

⇒ Best case : All DP table cells are calculated because these is no break.

$T(n,w) = O(nw)$

Conclusion: The Making Change problem was successfully implemented using Dynamic Programming. The algorithm stores the solutions of smaller subproblems in a DP table and uses them to find the required solution efficiently. It reduces repeated calculations and provides an efficient approach for finding the minimum number of coins or total number of ways to make a given amount.